TUTORIAL DE GIT – CAPITULO 3: TRABAJO CON REPOSITORIOS Y ARCHIVOS

NOMBRE: Jhon Mario Rodríguez Flechas

ASIGNATURA: Introducción a la ciencia de datos

CAPÍTULO 3. TRABAJO CON REPOSITORIOS Y ARCHIVOS

3.1. Introducción

Un repositorio Git permite almacenar los archivos de un proyecto y conservar un historial de los cambios realizados. Para trabajar de manera organizada, es importante conocer cómo crear repositorios, revisar su estado y administrar los archivos que contienen.

3.2. ¿Qué es un repositorio local?

Un repositorio local es aquel que se encuentra almacenado en el computador del usuario. Allí se pueden crear archivos, modificarlos, preparar cambios y registrar commits sin necesidad de enviar inmediatamente cada modificación a internet.

En esta actividad, la carpeta Actividad2-Git funciona como el repositorio local del proyecto.

3.3. ¿Cómo clonar un repositorio?

Clonar un repositorio consiste en crear una copia local de un repositorio remoto. Esto permite trabajar en el computador con los archivos y el historial del proyecto.

El comando utilizado es:

"git clone URL_DEL_REPOSITORIO"

La dirección debe reemplazarse por la URL real del repositorio que se desea copiar.

3.4. Consultar el estado de los archivos

El comando "git status" permite conocer qué archivos son nuevos, cuáles han sido modificados y cuáles están preparados para registrarse en un commit.

Ejemplo:

"git status"

Este comando es útil antes de registrar cambios, porque permite comprobar qué modificaciones se incluirán.

3.5. Agregar archivos al seguimiento de Git

El comando "git add" prepara los archivos para incluir sus cambios en el siguiente commit.

Para preparar un archivo específico se utiliza:

"git add nombre_archivo.txt"

Para preparar todos los cambios de la carpeta actual se puede utilizar:

"git add ."

Es recomendable revisar el estado del repositorio antes de preparar los archivos.

3.6. Eliminar archivos de un repositorio

Git permite registrar la eliminación de archivos. Si se desea eliminar un archivo del repositorio y preparar esa eliminación para el siguiente commit, se puede utilizar:

"git rm nombre_archivo.txt"

Este comando elimina el archivo del directorio de trabajo y prepara su eliminación para el próximo commit. Debe utilizarse con cuidado para no borrar información necesaria.

3.7. Consultar el historial de modificaciones

El comando "git log" muestra los commits realizados en el repositorio. Permite consultar sus identificadores, autores, fechas y mensajes.

Ejemplo:

"git log --oneline"

Esta variante presenta el historial de manera resumida, facilitando la revisión de los cambios registrados.

3.8. Organización de los archivos del proyecto

Un repositorio bien organizado facilita la búsqueda y mantenimiento de los documentos. Para esta actividad se utiliza la carpeta Tutorial-Git para guardar los capítulos del tutorial en Word y PDF, mientras que los archivos de texto y el README se mantienen en la carpeta principal del proyecto.

3.9. Aplicación en ciencia de datos

En proyectos de ciencia de datos es necesario organizar documentos, scripts, archivos de configuración y resultados de análisis. Git ayuda a mantener un historial de las modificaciones y facilita la identificación de los cambios realizados durante el desarrollo del proyecto.

3.10. Conclusión

La administración de repositorios y archivos permite mantener los proyectos organizados y controlar sus modificaciones. Comandos como "git clone", "git status", "git add", "git rm" y "git log" ayudan a gestionar el trabajo y revisar el historial de los archivos.

Referencia bibliográfica

Chacon, S., & Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book/es/v2